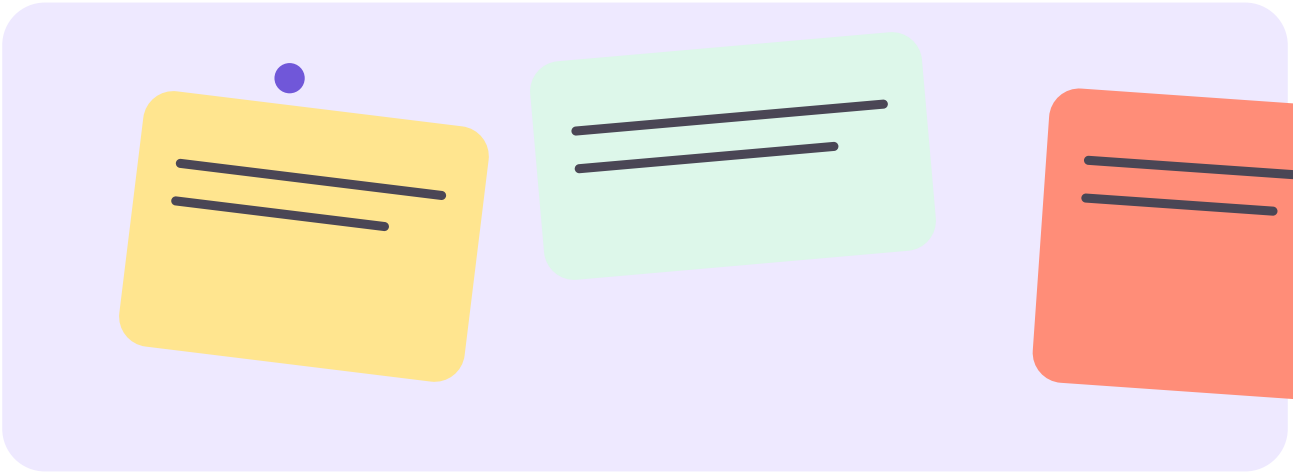
A playful, private space for everything you don’t want to forget

Product concept • feature specification • privacy architecture • MCP ecosystem • phased roadmap



Working title: intentionally undecided. This document describes the product without locking in a name.

The product starts as a lightweight thought-capture companion: a place to drop notes, links, screenshots and images, then find them again. It can grow into a personal media-memory and recommendation system, with MCP acting as a permissioned bridge to AI assistants—not as a requirement for using the app.

Contents

1. **Product vision and principles** — The problem, promise and product boundaries

2. **Core experience and feature catalogue** — Capture, organise, retrieve, themes and pet companion

3. **Privacy-first data architecture** — Local-first, encryption, sync choices and threat model

4. **MCP: useful integrations beyond chat** — Tools, resources, prompts and safe permission boundaries

5. **Part 2: Watchlist and taste profile** — Anime, films, genres, progress and recommendations

6. **Visual design and motion system** — Palette, themes, animation, accessibility and performance

7. **Technical architecture** — Go core, database, web app, extension and packaging

8. **Phase-wise roadmap and timeline** — Milestones, deliverables, dependencies and exit criteria

9. **Scope control and risks** — What to defer, risks and practical decisions

10. **MVP acceptance checklist** — What must work before calling v1 usable

Planning assumption: one developer working part-time alongside studies/work. The timeline is an estimate, not a commitment; integrations and platform-specific desktop work can move it.

1. Product vision and principles

Core promise: capture anything quickly, keep it under your control, and retrieve it when it matters. The product should feel like a friendly extension of the desktop—not another heavy productivity suite.

The product has two connected layers:

- **Personal capture layer:** quick notes, ideas, tasks, links, images, screenshots and saved snippets.
- **Personal context layer:** optional collections such as a watchlist, taste profile and other structured memories that can be queried by the user or an AI assistant.
- **Connection layer:** a local MCP server exposes only explicitly permitted data and actions to compatible AI clients.

Product principles

- **Capture before organisation.** One shortcut should be enough to save a thought. Classification can happen later.
- **Private by default.** The developer/operator should not be able to read user notes. No mandatory account or central note database.
- **Useful without AI.** Search, tags, collections and reminders must work even when no model or MCP client is connected.
- **Lightweight at idle.** Avoid Electron, background polling and always-running heavyweight webviews.
- **Playful, not distracting.** Themes and a small companion add personality but can be hidden or reduced.
- **Portable data.** Users can export their data in documented, non-proprietary formats.

Product boundaries

This is not intended to become a full project-management platform, social network, streaming service or AI chatbot. Keep the core small. Add modules only when they reinforce personal capture, retrieval or context.

2. Core experience and feature catalogue

2.1 Capture surfaces

- **Desktop quick capture:** global keyboard shortcut opens a tiny input window; save text in one action. Support title/body, note type, colour and optional collection.
- **System tray:** open quick capture, open dashboard, show recent notes, pause reminders, toggle pet, lock vault and quit.
- **Floating sticky notes:** optional always-on-top cards; drag, resize, pin, minimise and change colour. Each can be dismissed without deleting its content.
- **Web dashboard / PWA:** responsive inbox and library, usable offline for local data on the same device where supported.
- **Browser extension:** save current page, selected text plus source URL/title, image, screenshot or a quick annotation. Ask for a destination collection optionally.
- **Clipboard capture:** optional menu item to save current clipboard contents; never read clipboard continuously.

2.2 Note types and content

- **Quick thought, idea, task, reminder, link, image, quote/snippet, reference.** Type is metadata, not a rigid workflow.
- **Rich content:** plain text and Markdown, checklists, links, code snippets and multiple image attachments.
- **Source metadata:** created/updated timestamps, original URL, page title, capture method and optional tags.
- **Attachments:** images and screenshots stored as files with thumbnails; avoid embedding large binaries directly in SQLite.
- **Actions:** pin, edit, duplicate, archive, delete, restore, copy, share/export and move to collection.

2.3 Organising and retrieval

- **Inbox-first flow:** new captures land in Inbox. Users can organise later, or leave them there.
- **Collections/spaces:** e.g. Personal, College, Coding, Projects, Reading, Anime & Films. Users create their own.
- **Tags:** lightweight labels; allow multiple tags and tag search.
- **Search:** title/body search, filters by type, date, tag, collection, pinned and archived status. Start with SQLite FTS5.
- **Later:** snooze for 2 hours, tomorrow, next Monday or custom date; show a gentle reminder when due.
- **Memory lane:** optionally resurface a saved note from weeks/months ago. User controls frequency and can disable it.
- **Bulk actions:** multi-select, tag, move, archive, export and delete.

2.4 Pet / desktop companion

A small 2D character that reacts to useful events rather than demanding attention. It should be optional, low-resource and theme-aware.

- States: idle, blink, working, sleepy, note-saved celebration, inbox-cleared celebration and gentle reminder.
- Interactions: click to open quick actions, recent notes, random resurfaced note or quick capture.
- Settings: hide pet, reduce motion, mute sounds, choose size/position and disable reactions.
- Implementation: sprite sheets, CSS/vector animation or lightweight Lottie; no 3D engine.

2.5 Export, backup and portability

- Export all notes and metadata as JSON plus Markdown files; attachments copied into a predictable folder structure.
- Export a ZIP archive with a manifest and checksums. Provide an import/restore flow.
- Allow per-collection export and selected-note export.
- Document the database schema and export format so users are not trapped in the app.

2.6 Accessibility and usability

- Full keyboard navigation, visible focus, screen-reader labels and sensible tab order.
- Contrast-safe text in every theme; never encode status using colour alone.
- Reduced-motion setting, no essential information conveyed only by animation, and optional sound off by default.
- Responsive layouts for small screens; large tap targets on mobile/PWA.

3. Privacy-first data architecture

Important interpretation: “not storing the notes myself” should mean the product operator does not receive or hold readable user content. It should not mean the app has nowhere to save data. A private notes app still needs a storage location—ideally the user's own device or a storage account they control.

3.1 Recommended default: local-first, zero-knowledge by design

- **Local database:** SQLite on the user's device. Notes and metadata remain local by default.
- **Local attachments:** images/screenshots stored in an app-managed folder, with references in SQLite.
- **Encryption at rest:** encrypt the database and attachment files using a user-controlled vault key. Use a vetted library and OS secure storage for key wrapping; do not invent cryptography.
- **No mandatory cloud account:** the app works offline and does not send note content to your own servers.
- **Optional sync later:** support a user-owned destination (e.g. their own WebDAV/S3-compatible storage) or a carefully designed encrypted relay. Server sees ciphertext only.
- **Clear telemetry stance:** no note content, titles, URLs or attachment names in analytics or crash reports. Prefer opt-in diagnostics with redaction.

3.2 Storage and sync options

Option	Privacy / trade-off	Recommendation
Local-only vault	Strongest simple privacy; no cross-device sync unless user exports/transfers backup.	Ship first; ideal default.
User-controlled sync	Encrypted data stored in a destination the user owns. Setup and conflict handling are harder.	Best next step after stable local app.
Your hosted sync	You operate infrastructure. Even with encryption, metadata and availability become your responsibility.	Avoid for v1; only consider if users request it.
Plain cloud notes	Operator or provider may be able to read content; weakest fit for the stated privacy goal.	Do not use as default.

3.3 Security details and threat model

- Threats to consider: stolen/lost laptop, malware running as the user, compromised browser extension, accidental cloud exposure, weak backups, malicious MCP client and logs containing sensitive data.
- Vault lock: manual lock, lock on OS session lock, optional inactivity timeout. Quick capture while locked should either require unlock or use a clearly disclosed limited mode.
- Key handling: derive a vault key using a modern password KDF (e.g. Argon2id) with a unique salt; protect local key material with OS credential storage where possible. Use established authenticated encryption (e.g. AES-GCM or XChaCha20-Poly1305 via reputable libraries).
- Backups: encrypted by default; explain that losing the vault password may mean losing access. Provide recovery-key option with clear warnings.
- Browser extension: request minimal permissions; do not request broad site access unless required. Explicitly show what will be saved before capture where appropriate.
- MCP: local-only bind by default, authenticated client access, per-tool scopes, read-only mode default, explicit write permission and audit trail of actions.
- Data deletion: explain deletion from app, trash retention, backups and any user-configured sync destination. Secure deletion from SSDs cannot be guaranteed by ordinary file overwrite.

3.4 What privacy can and cannot promise

A local encrypted vault can keep the operator from reading stored content, but it cannot protect data from malware, a compromised OS account, a malicious AI client that has been granted access, or a user who shares decrypted exports. If an AI provider receives note text for a query, that text leaves the device and is governed by that provider's policies. Make this boundary visible and require user consent.

Recommended privacy statement: “Your notes live on your device by default. We do not operate a central readable-notes database. Optional sync is encrypted and user-controlled. When you ask an external AI service to process content, only the information you choose to share is sent to that service.”

4. MCP: useful integrations beyond chat

MCP should expose a safe interface to the user's own data, not become a reason to upload everything to a server. Start with a local MCP server that talks to the same application service and database. It should be optional: the app remains fully usable without it.

4.1 Initial MCP tools

Tool	Purpose	Safety / permission
create_note	Create a note, link, task or idea in a chosen collection.	Write permission; confirm collection and content.
search_notes	Search note text and metadata; return compact matches.	Read permission; limit results and redact attachments by default.
get_note	Fetch one note by ID, optionally including selected attachment metadata.	Read permission; user-controlled attachment access.
update_note	Edit title/body/tags/type/collection.	Write permission; return a diff/summary.
archive_note	Archive rather than permanently delete.	Write permission; permanent delete should be separate.
list_collections	List collection names and counts.	Read metadata permission.
save_link	Save URL, title and optional user-provided summary.	Write permission; don't fetch private pages silently.
get_recent_notes	Return recent notes for a user-requested recap.	Read permission; bounded time range.
get_due_items	Return snoozed reminders/tasks due now or soon.	Read permission; explicit time window.
create_collection	Create a collection for a new topic/project.	Write permission.

4.2 MCP resources and prompts

- **Resources:** `braindump://notes/recent`, `braindump://collections`, `braindump://stats`, `braindump://tasks/due`, `braindump://watchlist/summary` (the last belongs to the later watchlist module).
- **Prompts:** “Organise my inbox”, “Find related ideas”, “Summarise my notes this week”, “Find forgotten tasks”, “Turn these notes into a project outline”.
- Prompts should guide the model; they must not grant permissions. Tool scopes remain the actual security boundary.

4.3 Other high-value MCP use cases

- **Personal knowledge librarian:** find notes about a topic, surface duplicates and suggest tags. Model proposes; user approves bulk changes.
- **Idea connection finder:** discover related ideas across notes and show why they match, with links back to original notes.
- **Meeting/action memory:** save meeting decisions and action items; query “what did we agree about the API?” only if the user stored/imported that material.
- **Learning companion:** turn saved snippets into flashcards, revision questions or a short recap. Keep generated content separate from source notes.
- **Project context pack:** assemble selected notes, links and files into a temporary context bundle for a coding assistant. User chooses exactly which collection is exposed.
- **Reading queue assistant:** summarise saved articles from user-selected URLs, extract key points and save takeaways. Fetching external pages must be explicit.
- **Task triage:** classify inbox notes into suggested tasks, ideas and references; never silently move or delete content.

- **Watchlist recommender:** use genres, ratings, completion history and explicit likes/dislikes to recommend anime/films, with explanations.
- **Preference-aware assistant:** answer “recommend a short anime with mystery and comedy” using the user's saved profile, not a generic global profile.
- **Export assistant:** prepare a selected collection for Markdown/JSON export and explain what will be included before writing files.

4.4 MCP governance

- Default to local transport / local endpoint. Never expose an unauthenticated MCP server to the LAN or internet.
- Separate scopes: metadata read, note-content read, attachment read, create, edit, archive, permanent delete, watchlist read/write.
- Use least privilege. An assistant connected for recommendations may only need read access to the watchlist profile, not every private note.
- Offer a visible connection status and per-client permission screen. Provide revoke access and rotate credentials.
- Keep an action log (what tool, when, client, affected item ID) without copying sensitive note bodies into logs.

5. Part 2: Watchlist and taste profile

Treat this as a distinct second product module—possibly later promoted into a companion mobile app or standalone app experience. It should reuse the privacy, storage, theme and MCP foundations, but it should not delay the core capture MVP.

5.1 What users can track

- Films, anime, TV series and optionally manga/manhwa/manhua as separate media types.
- Status: planned, watching, on hold, completed, dropped, rewatching.
- Progress: season/episode for series; optional watch date and completion date.
- Personal rating, favourite flag, short review, memorable scenes/characters and private notes.
- Genres and tags: comedy, romance, fantasy, mystery, slice of life, action, thriller, drama, etc.
- Optional metadata: release year, format, episode count, language, source/service and poster image.
- Watchlist size can remain small and personal; support at least a handful to a few hundred entries without making it a social catalogue.

5.2 Taste profile

- Explicit preferences: favourite genres, disliked genres, preferred length, pacing, tone, romance/comedy balance and tolerance for cliffhangers.
- Implicit preferences: infer cautiously from completed titles, ratings, dropped titles and rewatch behaviour.
- User can inspect, edit, reset or disable inferred preferences. Never treat an inference as a fact.
- Separate “liked genre” from “watched genre”: watching a title does not automatically mean the user likes every genre attached to it.
- Track recommendation feedback: interested, not interested, already watched, not my taste. This improves future suggestions.

5.3 MCP recommendation workflow

- User asks: “Recommend a 12-episode rom-com anime with a wholesome tone.”
- MCP reads only the watchlist profile and relevant history if the user has granted that scope.
- Recommendation logic filters already-watched titles, respects dislikes and returns a short list with reasons tied to saved preferences.
- User can mark a title as planned or reject it; the assistant writes the decision only after explicit user instruction.
- Optional: expose `get_watchlist`, `get_taste_profile`, `add_to_watchlist`, `update_watch_progress`, `rate_title`, `record_recommendation_feedback`.

5.4 Recommendation engine strategy

- **Start rule-based:** genre overlap, tone, length, status exclusion and user ratings. Easy to explain and works without a hosted model.
- **Then add embeddings:** semantic similarity over descriptions and user-written reviews, computed locally if practical.
- **LLM as an explainer:** use MCP-connected assistants to explain why a title fits. Do not make the LLM the sole source of catalogue facts.
- **Catalogue metadata:** if an external metadata provider is used, respect its API terms, attribution and rate limits. Do not scrape services that prohibit it.
- **Privacy:** keep taste profile local; send only the minimum profile fields needed for an external recommendation request, after consent.

5.5 App direction

A future mobile app could focus on watchlist capture, progress updates, discovery and recommendations. The desktop companion remains the fast capture surface. Both should share a documented data model and encrypted sync option rather than maintaining separate databases.

6. Visual design and motion system

The visual direction should feel expressive and tactile: a clean canvas, confident typography, thick outlines, soft shadows and bright accent colours. Neobrutalism is the default identity, while themes can completely change the atmosphere without changing the information architecture.

6.1 Default palette: playful neobrutalism

Role	Colour	Use
Ink	#20212B	Main text, outlines, icons.
Canvas	#FFFDF8	Warm off-white background.
Electric violet	#7057D9	Primary actions, active states.
Mint	#DDF7EA	Positive cards, completed states.
Coral	#FF8D78	Highlights, playful accents.
Butter yellow	#FFE58F	Pinned notes, reminders.
Soft blue	#DCEBFF	Links, informational cards.

6.2 Theme packs

Theme	Visual language	Companion / motion
Default brutal	Warm paper, black borders, offset shadows, violet accents.	Friendly blob; pop/bounce micro-interactions.
Coder / terminal	Dark graphite, terminal green/cyan, monospace details.	Tiny robot; scanline/glow used sparingly.
Cutesy pastel	Lavender, peach, cream, rounded cards and soft gradients.	Bunny/cloud pet; gentle float and blink.
Anime-inspired	High-contrast editorial panels, expressive accent colours, optional wallpaper.	Neko or mascot sprite; expressive idle states.
Focus / minimal	Muted monochrome, reduced borders and visual noise.	Pet hidden or very subtle.

6.3 Motion language

- Use motion to communicate state: note saved, panel opened, card moved, filter applied and reminder surfaced.
- Micro-interactions: 120–220 ms for hover/press, 180–300 ms for panel transitions, 250–450 ms for celebratory pet reactions.
- Use spring-like easing for cards and pet motion; avoid making the entire interface constantly move.
- Transitions should preserve context and avoid layout jumps. Prefer transform/opacity over expensive blur and large animated shadows.
- Use skeletons only when content genuinely loads; show immediate local content whenever possible.
- Respect `prefers-reduced-motion`; provide a global reduced-motion setting and disable ambient animations when idle.

6.4 Signature interactions

- Quick-capture card appears like a sticky note sliding onto the desktop; saving gives a small satisfying stamp/check animation.
- Drag a note into a collection and see a clear destination preview.
- Pet reacts briefly when a note is captured, then returns to idle.
- Theme switch transitions palette, typography, borders and pet together—no partial theme mismatch.

- Empty inbox has a warm, playful illustration and a single obvious capture action.

6.5 Performance and polish

Use a small number of reusable motion primitives, lazy-load theme assets, optimise sprite sheets and avoid autoplay audio. Animation should never make typing, scrolling or capture feel slower. The UI should remain snappy on ordinary integrated graphics and lower-power laptops.

7. Technical architecture

Recommended direction: Go for the resident desktop/core service, SQLite for local structured data, a lightweight React web UI for the dashboard, and TypeScript for the browser extension. This balances learning value, implementation speed and a small idle footprint.

7.1 High-level architecture

Component	Responsibility	Notes
Go core daemon	Tray, global shortcut, quick-capture window bridge, local API, vault lifecycle, reminders.	Event-driven; no busy polling. Start with Windows.
SQLite vault	Notes, collections, tags, watchlist metadata, settings and FTS index.	Encrypted database; migrations; WAL where appropriate.
Attachment store	Images, screenshots, thumbnails and exported files.	Content-addressed or UUID filenames; encrypted at rest.
Web dashboard	Inbox, library, collections, search, settings and later watchlist UI.	React + Vite is sufficient; PWA later.
Browser extension	Capture page, selection, image and screenshot.	Minimal permissions; explicit capture UX.
MCP server	Expose scoped tools/resources/prompts to AI clients.	Same service layer; local-only default.
Optional sync	Encrypted backup and cross-device replication.	Not in MVP; design data format for it.

7.2 Local communication

- Desktop UI and extension talk to the Go service through a loopback-only local HTTP API with a random per-install token, or use native messaging for the extension.
- Bind only to 127.0.0.1; validate Origin/Host and authenticate requests. Do not trust arbitrary websites to call the local API.
- Native Messaging is a stronger extension-to-host boundary but adds installation complexity. A localhost API is simpler for an early prototype; harden it before public release.
- Use one application/service layer for UI, extension and MCP so validation and permissions are not duplicated.

7.3 Suggested data model

Entity	Key fields (illustrative)
notes	id, title, body, type, status, collection_id, pinned, created_at, updated_at, archived_at, snoozed_until
collections	id, name, colour, icon, sort_order, created_at
tags / note_tags	tags: id, name; join: note_id, tag_id
attachments	id, note_id, relative_path, mime_type, size, checksum, created_at
watch_items	id, title, media_type, status, progress, rating, release_year, external_ref, created_at, updated_at
watch_genres	watch_item_id, genre; plus user genre preferences and exclusions
mcp_clients / permissions	client_id, scopes, created_at, last_used_at, revoked_at; store token hashes, not raw tokens
audit_events	id, client_id, action, target_id, timestamp, outcome; avoid content payloads

7.4 Suggested repository layout

```
/cmd/desktop/ Go app entry point
/cmd/mcp/ MCP server entry point (or same binary subcommand)
/internal/app/ application services and use cases
/internal/store/ SQLite repositories and migrations
/internal/vault/ encryption, lock/unlock, key lifecycle
/internal/desktop/ tray, shortcut, native window integration
/internal/http/ loopback API and auth middleware
/internal/mcp/ tools, resources, prompts and scopes
/internal/sync/ future sync interfaces (not active in MVP)
/web/ React dashboard
/extension/ browser extension
/assets/ themes, sprites, icons and sound effects
/docs/ architecture, threat model, export format
```

7.5 Resource goals

- Target near-zero CPU while idle; do not poll the database or animate continuously in the background.
- Keep the resident process small; dashboard/webview should load only when opened. Actual RAM depends on native toolkit, browser/webview and OS.
- Use image thumbnails and lazy loading. Cap thumbnail dimensions and avoid decoding full-resolution images in list views.
- Measure startup time, idle RAM, idle CPU, quick-capture latency and database query time on target hardware. Treat targets as benchmarks, not promises.
- Do not choose a UI toolkit solely on theoretical memory use; build a small spike and measure Windows packaging, global shortcut reliability and window behaviour.

8. Phase-wise roadmap and timeline

A realistic first release is approximately 10–12 weeks part-time, assuming one developer and a Windows-first build. The watchlist module is a separate follow-on phase. Add buffer if desktop native integration or encryption is new territory.

Phase	Timeline	Scope and deliverables	Exit criteria
0 — Product & technical spikes	Week 1	Confirm Windows-first target, choose desktop window/tray approach, prototype global shortcut, test Go SQLite layer, test encrypted DB/file approach, sketch UI.	Can open quick-capture window from shortcut; encryption approach validated; architecture decision recorded.
1 — Private capture MVP	Weeks 2–4	Go core, tray, quick capture, SQLite vault, create/edit/archive notes, collections, tags, attachments, search, lock/unlock, export/import.	Offline capture and retrieval work; data stays local; export/restore tested; no central note storage.
2 — Dashboard & personality	Weeks 5–6	Responsive dashboard/PWA shell, inbox/library, filters, pinned notes, sticky-note cards, default neobrutalism theme, theme engine, pet prototype, reduced-motion settings.	User can manage notes without opening the quick-capture popup; theme switch is coherent; pet can be hidden.
3 — Browser extension	Weeks 7–8	Save page, selected text with source, image, quick note, send to collection; local service auth; permissions UX; Chrome first.	Capture flows reliably; extension cannot write without local auth; permissions are minimal and documented.
4 — MCP v1	Weeks 9–10	Local MCP server, read/search tools, create/update/archive with scopes, resources, prompts, client connection and revoke UI.	One supported AI client can search and create notes only within granted permissions; audit events recorded.
5 — Hardening & v1 release	Weeks 11–12	Installer, auto-start opt-in, backup/restore, crash handling, performance checks, accessibility pass, documentation, privacy notice and release packaging.	Clean install/uninstall; recovery path tested; threat-model checklist reviewed; no known critical data-loss bugs.
6 — Watchlist module	Weeks 13–16	Media library, status/progress, genres, ratings, taste profile, recommendation feedback, MCP watchlist tools/resources and basic rule-based recommendations.	Can track films/anime, exclude watched titles and produce explainable preference-aware suggestions.
7 — Mobile companion / sync	Weeks 17–22+	Evaluate mobile app, user-controlled encrypted sync, conflict resolution, notification support and shared data model.	Two devices can sync without exposing readable data to operator; conflict and recovery tests pass.

Suggested weekly rhythm

- Build in vertical slices: one working user flow at a time, not separate weeks of backend-only work.
- Keep a demoable build at the end of every phase. Use real local data and test restore early.
- Reserve roughly 20% of time for integration bugs, Windows packaging, accessibility and polish.
- Do not begin watchlist or cloud sync until core capture, encryption, export and restore are stable.

Timeline at a glance

Weeks	1	2–4	5–6	7–8	9–10	11–12	13–16	17+
Workstream	Spike	Core MVP	UI/theme	Extension	MCP	Release	Watchlist	Mobile/sync

9. Scope control, risks and open decisions

9.1 Keep out of the first release

- Cloud accounts, hosted sync, social sharing and collaborative editing.
- AI auto-tagging, auto-summarisation and semantic search before basic search is dependable.
- Theme marketplace and user-authored theme scripting.
- Full mobile app, native macOS/Linux support and multiple browser engines at once.
- Complex OCR, voice transcription, recurring task management or calendar replacement.
- Public recommendation feed, ratings community or scraping streaming catalogues.

9.2 Main risks and mitigations

Risk	Why it matters	Mitigation
Desktop integration complexity	Tray, global shortcuts, popup windows and installers vary by OS.	Windows-first; prove native integration in Phase 0 before building the full UI.
Encryption mistakes	A privacy promise is only as good as key handling and recovery.	Use established libraries; threat-model; independent review before claiming strong security.
MCP overreach	An AI client could read or alter more than intended.	Least-privilege scopes, local auth, per-client revocation and explicit write grants.
Feature creep	Watchlist, pet, extension, sync and themes can overwhelm the MVP.	Phase gates; protect the core capture loop; watchlist is Part 2.
Heavy UI / animation	Constant effects and webviews can undermine lightweight goal.	Lazy-load dashboard/assets; no ambient motion by default; benchmark idle footprint.
Catalogue data accuracy	Recommendations become untrustworthy with stale or scraped metadata.	Use permitted metadata sources; show source and allow corrections; separate facts from AI guesses.

9.3 Decisions to make before implementation

- **Platform:** recommended Windows first, cross-platform architecture later.
- **Storage:** recommended local-first encrypted vault; user-controlled sync later.
- **Desktop UI:** recommended lightweight native capture/tray plus web dashboard loaded on demand. Evaluate Wails only if one integrated desktop window becomes more important than idle footprint.
- **Extension transport:** prototype localhost API vs Native Messaging; choose based on security and installation friction.
- **Vault UX:** decide password, OS-account-bound key, recovery key and lock timeout.
- **Watchlist scope:** films + anime first; decide later whether to include manga/manhwa/manhua and TV.
- **Brand/name:** intentionally open. Keep package IDs and repository naming neutral until selected.

9.4 Product success metrics

- Capture latency: time from shortcut to saved note; aim for a near-instant interaction.
- Retrieval success: users can find a note using search, tags or collection without remembering where it was saved.
- Reliability: no data loss in crash/restart, backup/restore and upgrade tests.
- Privacy: no note content leaves device unless the user explicitly invokes a feature that sends it to an external service.
- Resource use: measure idle CPU/RAM, startup time and attachment-list performance.
- Delight: users keep the pet/theme enabled because it adds charm, not because it blocks core tasks.

10. MVP acceptance checklist

The MVP is ready when these are true—not merely when the screens exist.

- Install and launch on a clean Windows account.
- Tray icon appears; quick-capture shortcut works and can be changed.
- Create, edit, pin, archive, restore and permanently delete a note.
- Create collections and tags; move notes between collections.
- Save and view multiple image attachments.
- Search titles and note bodies; filter by collection, type and date.
- Lock and unlock the local vault; verify stored content is encrypted at rest.
- Export a complete archive and successfully restore it on a fresh profile.
- Use the app offline; no mandatory account or hosted note storage.
- Dashboard loads on demand; idle process does not continuously consume CPU.
- Browser extension (when included) saves a page and selected text with source metadata.
- MCP read/write scopes are independently configurable and revocable.
- External AI sharing is explicit and clearly identified to the user.
- Theme switch works; reduced-motion mode works; pet can be hidden.
- Privacy notice explains local storage, backups, external AI processing and limitations.

Recommended first build target

Ship a private, local-first Windows capture app before building the complete ecosystem. The first satisfying milestone is: press a shortcut, jot a thought, save an image, close the popup, and find the item later in the dashboard. Then add personality, extension capture and MCP. The watchlist is a deliberate Part 2, with its own UI and recommendation loop.

Working product document • Name intentionally undecided • Roadmap estimate: 10–12 weeks to core v1, followed by watchlist and mobile/sync phases.